

CKBuilder Handbook

Welcome to the CKBuilder Handbook! This is a resource for all cohort members participating on the [Community Keeps Building](#) Builders' track.

Here, you'll find quick links for

Please note: some documents may refer to Lumos or Capsule for performing tasks. These tools are now deprecated and the best tools are mentioned below.

How to use this Handbook

This Handbook will guide you through learning and building on CKB, providing you with important resources that will help structure your study: starting from the basics, towards more advanced concepts.

The CKBuilder programme is designed to be something you can do on evenings or weekends, with a minimum expected commitment of 4-5 hours per week. Many people will put in more time than this, but what's important is that you work at a steady pace that allows you time to absorb what you study. **If you have any questions:** please contact the programme director (Neon).

Reporting standards

As you work through the documentation and tooling, **it is essential to start your own dev log on GitHub to record your learning and progress**. This is required irrespective of whether you are studying docs or building an application. Below are some reporting requirements that help to ensure everyone follows the same procedure. Here is an [example report](#).

Reimbursement is pro-rata and tied directly to your reporting log: if you miss out on reporting it will result in reduced reimbursement proportional to the number of weeks missed. Reports must be:

Once per week. Pick a day and submit your report every week on that day.

Contemporaneous. Each published report must reflect the week just completed. You should not compile reports for all weeks at the end of the month. Multiple reports published at once to cover multiple weeks will count as just one report and reimbursed accordingly.

Published on GitHub in your personal CKBuilder repository.

An accurate reflection of your progress. Use your log as an opportunity to share what you learned, what you found interesting or challenging, and what you plan to work on moving forward. Include screenshots and other evidence of progress. You can use AI to tidy up your writing, but avoid using AI to generate your reports.

Introduction

[Introduction to Nervos CKB](#) - basic technical concepts and terminology explained

[Getting started on CKB](#) -

[Quick start](#) - set up your dev environment and project quickly using [OffCKB](#). Further information on networks and RPCs are available from this page.

[CKB Academy lessons 1 and 2](#) - further learning to build your theoretical knowledge

[Introduction to Script \(smart contracts\)](#) - a introduction to smart contracts on CKB

You can also view some community-developed manuals to support your learning here:

[Learning CKB - 24 lessons across 5 phases](#) by Jnr.bit

[Learn CKB in 45 mins](#) by truthixify

For a more detailed dive into the concepts and theory of CKB, a reading list is provided at the end of this document. This should be studied in your own time and is not a requirement prior to starting the beginner exercises.

Beginner

Basic exercises - once you have set up your environment and understood the technical concepts, get started on these tutorials:

Please note: for all tutorials and practical exercises, please remember to retain screenshots as proof of participation and completion and record on your dev log

[Transfer CKB](#)

[Store Data on Cell](#)

[Create Fungible Token](#)

[Create DOB \(digital object\)](#)

[Build a Simple Lock](#)

The [L1 developer course](#) is another resource to understand the theoretical aspects of CKB. Many Lab exercises here use Lumos or Capsule which you can ignore.

Learn building on CKB (Javascript, Typescript), using [CCC](#) (Common Chain Connector), a beginner-friendly all-in-one tool to build applications with integrated support for major wallets:

Explore [CCC App](#)

Test your code in the [CCC Playground](#)

[Code examples](#)

[API](#)

Note: For intensive smart contract development on CKB, it is **recommended to learn Rust**. Rust is well known for its efficiency and security and has strong tooling support on CKB.

[Rust SDK](#)

[CKB-CLI](#) - a CKB command-line tool developed in Rust

For other programming languages:

[Go](#)

[Java](#)

Other devtools, including:

[Testnet faucet](#)

[CKB Debugger](#)

[CKB Tools](#) website

Payment channels on CKB

CKB is designed to be highly secure and decentralised at its base layer. It is optimised for verification. At the same time, it supports scaling and high-throughput use cases on layers built on top. The most promising solutions are around payment channel technologies. On CKB, the main payment channels are **Fiber Network** and **Perun**.

[Fiber Network](#) - a Lightning-compatible p2p payment channel/swap network on CKB

[Documentation](#)

[Showcase](#) - open source community projects on Fiber

[Perun](#) - Ethereum-compatible p2p payment channel/swaps on CKB

Task: create your own basic application

Now you can put what you've learned above to the test!

Use your previous study and the tools above to create a basic application. It could, for example, allow users to generate their own tokens or mint spores (dobs). Alternatively, it could involve payment processing using CKB's payment channels. Discuss your ideas with the programme

director (Neon) and with members of CKB DevRel, they will be able to help guide you. If you have some development background already, you may wish to be more adventurous.

To aid your building, we also have a [CKB AI scholarship initiative](#) which gives you access to subscription-grade AI models. Please note that this is a shared resource. Avoid unnecessary use to prevent reaching limits which may affect other users. You can also connect your AI to [CKB AI MCP](#) for better context during development.

There is always the chance a good idea opens the door to funding from the [Spark programme](#) or the [CKB Community Fund DAO](#)!

Further reading

Deepen your understanding of CKB using this collection of core documents:

[GitHub - nervosnetwork/rfcs: This repository contains proposals, standards and documentations related to Nervos Network.](#)

Intermediate

Want to further your understanding of CKB Development? Here are some modules to get you going:

Script development course

[Class 1: Validation Model](#)

[Class 2: Script Basics](#)

[Class 3: UDT](#)

[Class 4: WebAssembly on CKB](#)

[Class 5: Debugging](#)

[Class 6: Type ID](#)

[Class 7: Advanced Duktape Examples](#)

[Class 8: Performant WASM](#)

[Class 9: Cycle Reductions in Duktape Script](#)

[Class 10: Language Choices](#)

[Detailed Rust scripting](#)

[Detailed JS scripting](#)

[Ecosystem scripts and libraries](#)

[Understanding Molecule and serialization](#)

Simple UDTs (sUDT)

A standard that defines the most basic implementation of a UDT (user defined token) fungible token on Nervos. An sUDT on Nervos is the equivalent of Ethereum tokens standards ERC20 or ERC777.

[Standards document](#)
[Tutorial](#)

Nervos DAO

Nervos DAO enable users to lock CKBytes to get compensation from Nervos CKB secondary issuance. This process is similar to staking on other platforms. Nervos DAO provides a "virtual hardcap" for CKByte holders to insulate them from inflation.

[Introduction](#)
[Deposit and Withdraw in Nervos DAO](#)
[Source code](#)
[NervDAO - a browser implementation](#)

Spore (DOBs)

Spore Protocol infuses digital objects (DOBs) with enduring value backed by tokenomics, redeemable at any time. Ensures true on-chain ownership, privacy, creative freedom and frictionless interaction.

[Spore protocol introduction](#)
[Comprehensive Spore documentation and tutorials](#)
[DOB Cookbook](#)
[How-to Recipes](#)
[SDK](#), [Examples](#), [Demos](#), [Contracts](#)
[DOB Decoder Standalone Server](#)

Advanced

If you want to challenge yourself further and really grow your skills, study the following resources.

Script-Sourced Rich Information (SSRI)

SSRI is a protocol for strong binding of relevant information and conventions to the Script itself on CKB. It allows developers to embed arbitrary information/logic into the Script, describing its behavior and managing data to off-chain applications.

[Introduction](#)

[SDK for implementing SSRI-compliant smart contracts on CKB](#)

[First prototype of SSRI-Compliant contract](#)

[Server for calling SSRI methods](#)

[Further Documentation](#)

[Pausable UDT: The first fully SSRI compliant and production ready contract that exemplifies all use cases that SSRI protocol covers.](#)

[Audit report](#)

RGB++ - Bitcoin's Turing Catalyst

RGB++ is an **asset issuance protocol for Bitcoin that leverages the flexibility and programmability of CKB** to enable dynamic use cases for Bitcoin assets. It unlocks the full potential of Bitcoin for a rich application ecosystem without altering its foundational security. This production-ready protocol offers a trustless method for dApps to interact directly with native Bitcoin transactions, facilitated by a lightweight, decentralized verifier. Website: www.rgbpp.com

[Introduction](#)

[RGB++ Protocol Light Paper](#)

[RGB++ Resource Hub](#), including Script Standard, CKB Bitcoin SPV, API service for asset management, API endpoints and development tools

[RGB++ SDK](#)

[RGB++ Explorer](#)

xUDT - Extensible User Defined Token standard

xUDT (Extensible User-Defined Token) is a token standard for creating and managing fungible tokens on Nervos CKB. xUDT builds upon the foundational Simple User-Defined Token (sUDT), which uses a minimal, predefined validation model. In contrast, xUDT introduces extensibility by allowing developers to attach custom validation logic via external Scripts — enabling more advanced governance, minting rules, and token behaviors.

[Introduction](#)

[How xUDT works](#)

[Enhance sUDT's Programmability with xUDT](#)

[xUDT RFC](#)

iCKB

iCKB is a protocol that aims to solve the illiquidity problem of the NervosDAO by tokenizing NervosDAO deposits into a liquid iCKB token. Website: www.ickb.org

[Whitepaper](#)

[Github repository](#)

[Audit report](#)

[iCKB SDK](#)

[Unlocking CKB Liquidity: A Deep Dive into the iCKB Protocol](#)

Reading list: Understanding CKB

- [Nervos Nation videos](#) explaining core CKB concepts visually at a high level, including UTXO/POW, RISC-V, Cell Model, RGB++, and more
- [Nervos: An In-Depth Overview of a Blockchain Network Built for Modularity](#)
- [Common Knowledge Base \(CKB\): Understanding Our Ethos](#)
- [A Deep Dive Into the Tokenomics of Nervos Network](#)
- [Where we're going, we don't need Accounts: the Future of Onboarding](#)
- [Common Knowledge Base \(CKB\): a Blockchain Developer's Dream](#)
- [Bitcoin vs. CKB: Two Approaches to Achieving Sustainable Security](#)
- [Web5: Extra Decentralized](#)
- [Comparing Blockchain Virtual Machines \(VMs\): EVM, WASM, SVM, and CKB-VM](#)